

UNIVERSIDADE FEDERAL DE SANTA MARIA
CENTRO DE TECNOLOGIA
CURSO DE CIÊNCIA DA COMPUTAÇÃO

Trabalho 1: Implementação de uma Aplicação Chat

Sistemas Distribuídos utilizando Java, gRPC e Protobuf

Disciplina: Sistemas Distribuídos

Davi de Castro Machado
Arthur Bogacki

Docente: Raul Ceretta Nunes

Santa Maria, RS
Maio, 2026

1. INTRODUÇÃO

O trabalho consiste na implementação de uma aplicação distribuída de Chat com suporte a uma única sala (room do servidor). Para a construção do sistema, foram utilizadas as tecnologias **Java 17**, **gRPC** e **Protocol Buffers** (Protobuf) para a definição de contratos de interface.

A arquitetura da solução baseia-se em dois tipos de processos distintos:

- **ServerChat (Servidor):** Responsável pela gestão dos usuários e distribuição das mensagens.
- **ClientChat (Cliente):** Interface que permite aos usuários interagir com o sistema.

O foco do projeto é garantir a interoperabilidade, a conformidade rigorosa com o contrato definido em `contrato-chat.proto` e a exploração de diferentes modelos de comunicação, como Unary e Server Streaming.

2. DESENVOLVIMENTO E RFAS

Abaixo são apresentados os detalhes da implementação, instruções de build e a validação dos Requisitos Funcionais Absolutos (RFAs).

Build e Execução

O projeto utiliza o Maven para automação de build e geração de fontes Protobuf.

```
# Compilar o projeto
mvn compile

# Iniciar o servidor de chat
mvn "exec:java" "-Dexec.mainClass=kiri.chat.Main" "-Dexec.args=server 50051"

# Iniciar um cliente (exemplo: alice)
mvn "exec:java" "-Dexec.mainClass=kiri.chat.Main" "-Dexec.args=client alice localhost 50051"
```

Uso da Aplicação

- **Modo:** O usuário deve escolher entre `server` ou `client`.
- **Comandos:**
 - `/list` : Exibe a lista de usuários online (via mensagem privada do servidor).
 - `/exit` : Encerra a sessão e notifica os demais usuários.

Matriz de Requisitos Funcionais Absolutos (RFAs)

RFA	DESCRIÇÃO	TESTE AUTOMÁTICO	TESTE MANUAL
RFA01	Registro com nome único; rejeita duplicatas	<code>registerRejectsDuplicateUsers</code>	Tentar conectar dois clientes com o mesmo <i>username</i> .
RFA02	Uma única sala criada pelo servidor	Implícito nos testes de integração	Validar que todos os clientes veem as mesmas mensagens.
RFA03	Broadcast de mensagens a todos	<code>receiveMessagesStreamsBroadcastsForRegisteredUser</code>	Enviar mensagem de um cliente e validar recepção nos outros.
RFA04	ChatMessage com from, content e timestamp	<code>chatMessageContainsTimestamp</code>	Verificar a presença de <i>timestamp</i> válido nas mensagens.
RFA05	Stream contínuo (Tempo Real)	<code>receiveMessagesStreamsBroadcastsForRegisteredUser</code>	Observar a chegada imediata de mensagens.
RFA06	Ordem por remetente preservada	<code>messagesPreserveOrderBySender</code>	Enviar sequência rápida e validar ordem de recepção.

RFA	DESCRIÇÃO	TESTE AUTOMÁTICO	TESTE MANUAL
RFA07	Notificações SYSTEM de entrada/saída	<code>leaveBroadcastsToRemainingSessions</code>	Validar mensagens SYSTEM ao conectar/ desconectar.

3. CONCLUSÃO E CONSIDERAÇÕES

A implementação deste projeto consolidou o entendimento sobre sistemas distribuídos modernos e o uso de chamadas de procedimento remoto (RPC). Através do **gRPC**, foi possível estabelecer uma comunicação eficiente e tipada, superando as complexidades de manipulação manual de bytes ou parsing de strings comumente encontrados em Sockets TCP.

A utilização de **Server Streaming** para o recebimento de mensagens provou ser a solução ideal para manter a baixa latência, permitindo que o servidor entregue dados em tempo real sem a necessidade de *polling* constante por parte dos clientes. A integração com **JUnit** para a validação automática dos RFAs garantiu que critérios críticos, como a unicidade de nomes e a preservação da ordem das mensagens, fossem respeitados em cada alteração do código.

Em resumo, a aplicação cumpre todos os requisitos propostos, demonstrando como ferramentas de IDL (Interface Definition Language) como o Protobuf facilitam a construção de sistemas interoperáveis e robustos.